كلية العلوم التطبيقية - ورزازات
ⵜⴰⵖⵡⵍⴰⵏⵜ ⵏ ⵜⵣⵔⴼⵜ - ⵓⵔⵣⴰⵣⴰⵜ
FACULTE DES SCIENCES APPLIQUEES - OUARZAZATE

Reconnaissance des caractères manuscrits amazighs à l'aide d'une architecture LeNet-5 adaptée aux images RGB

ES-SABBAR Ismail
Étudiant en Intelligence Artificielle et Applications
Département d'Informatique, Faculté des Sciences Appliquées d'Ouarzazate
Université Ibn Zohr, Ouarzazate, Maroc
ismail.essabbar.19@edu.uiz.ac.ma
Travail encadré par : Mohamed BENADDY

***Résumé*—La reconnaissance automatique des caractères manuscrits en Tifinagh demeure délicate en raison des variations d'écriture et des ressemblances géométriques entre certaines classes. Cet article présente une adaptation du réseau convolutif LeNet-5 au traitement d'images RGB de la base AMHCD (25 740 images, 33 classes). Les données sont réparties de façon stratifiée en ensembles d'entraînement, de validation et de test (70/15/15). Deux stratégies d'optimisation, Adam et SGD avec momentum, sont comparées selon un même protocole de 20 époques. Sur le jeu de test, SGD atteint une exactitude de 97,95 % contre 97,38 % pour Adam. Les courbes d'apprentissage, les matrices de confusion disponibles et l'analyse visuelle des erreurs montrent l'intérêt de cette architecture compacte, tout en soulignant les confusions possibles entre graphèmes de formes voisines. La discussion distingue les observations rapportées des limites expérimentales et des expériences complémentaires à mener.**

***Index Terms*—Tifinagh, AMHCD, LeNet-5, réseaux de neurones convolutifs, reconnaissance manuscrite, classification multiclasse, Adam, SGD.**

I. Introduction

La reconnaissance des caractères manuscrits (*Handwritten Character Recognition*, HCR) constitue un enjeu central du traitement automatique des documents. Les variations de forme, d'épaisseur, d'orientation et de tracé compliquent la correspondance entre une image et le symbole qu'elle représente. Pour le Tifinagh, ces difficultés sont accentuées par la proximité visuelle de plusieurs graphèmes, notamment lorsque l'écriture est peu régulière [3], [4].

Les réseaux de neurones convolutifs (*Convolutional Neural Networks*, CNN) permettent d'apprendre des représentations visuelles sans imposer une extraction manuelle de descripteurs. LeNet-5, introduit par LeCun et ses collaborateurs, demeure une architecture de référence pour étudier la classification de petits caractères manuscrits [1], [2]. Sa structure relativement légère facilite également son évaluation sur processeur.

Le présent travail examine la capacité d'une variante RGB de LeNet-5 à reconnaître les 33 catégories de la base AMHCD. L'objectif n'est pas de proposer une nouvelle famille de réseaux, mais de documenter une mise en œuvre reproductible, d'en comparer deux optimiseurs et d'identifier les principales sources d'erreur.

II. Travaux connexes et contributions

Les premières approches de reconnaissance du Tifinagh associaient des descripteurs conçus manuellement à des classifieurs tels que les SVM ou les k plus proches voisins. Les travaux basés sur les CNN ont ensuite montré l'intérêt des caractéristiques apprises pour cette écriture [4]. Notre étude s'inscrit dans cette continuité, avec une architecture fondée sur LeNet-5 plutôt qu'un réseau très profond.

Les contributions de l'étude sont les suivantes : (i) adaptation de LeNet-5 à une entrée RGB de 32×32 pixels et à 33 sorties ; (ii) comparaison d'Adam et de SGD avec momentum sur un découpage commun ; (iii) analyse quantitative par exactitude, précision, rappel et score F1 macro ; (iv) interprétation qualitative au moyen de prédictions, de confusions et de cartes d'activation.

III. Fondements théoriques et enjeux de la reconnaissance

A. Du tracé manuscrit à l'étiquette de classe

La classification d'un caractère isolé cherche à associer à chaque image une étiquette parmi un vocabulaire fini. L'image contient cependant beaucoup plus d'information que le seul symbole : épaisseur du trait, inclinaison, position dans le cadre, couleur, contraste et particularités propres au rédacteur. Deux réalisations d'une même lettre peuvent ainsi avoir des apparences sensiblement différentes. Inversement, deux lettres distinctes peuvent se ressembler lorsqu'un segment est abrégé, effacé ou déformé.

Cette distinction entre *variabilité intra-classe* et *similarité inter-classe* est déterminante pour le Tifinagh. Certaines classes reposent sur de faibles différences géométriques, par exemple l'ajout d'un trait ou la fermeture d'un contour. Un

système de reconnaissance doit donc tolérer les variations qui ne modifient pas l'identité du caractère, tout en restant sensible aux petits détails qui permettent de différencier deux étiquettes.

B. Intérêt des caractéristiques apprises

Une chaîne de classification classique peut extraire des indicateurs prédéfinis (nombre de contours, projections horizontales et verticales, moments géométriques) avant d'utiliser un classifieur tel qu'un SVM ou une méthode de voisins proches. Ces approches dépendent directement de la pertinence de caractéristiques construites à l'avance. Elles constituent des solutions possibles, mais ne sont pas évaluées expérimentalement dans ce travail.

Un CNN apprend, au contraire, les filtres de représentation en même temps que les paramètres de décision. Ses premières couches extraient des motifs locaux ; les suivantes agrègent des indices plus complexes. Les cartes d'activation produites par les convolutions ne correspondent pas nécessairement à des éléments linguistiques clairement nommables : leur rôle est d'améliorer la séparation des catégories pendant l'apprentissage [1], [2].

C. Pourquoi s'appuyer sur LeNet-5 ?

LeNet-5 offre un compromis intéressant entre la simplicité du réseau et la possibilité d'apprendre plusieurs niveaux de caractéristiques. Son cheminement associant convolutions et réductions de résolution convient à des entrées de petite taille. À la différence de la configuration originelle destinée à des images monochromes, la présente implémentation reçoit trois canaux et prédit 33 classes. Ces adaptations sont nécessaires pour correspondre à la représentation d'entrée et aux étiquettes AMHCD.

L'emploi d'une architecture historique ne signifie pas qu'elle surpasse des modèles modernes. L'objectif de l'étude est plus circonscrit : documenter une base expérimentale simple, interprétable et exécutable sur CPU. Une comparaison avec un réseau profond ou avec une entrée en niveaux de gris constituerait une étude différente.

D. Question et hypothèses expérimentales

La question centrale est la suivante : *dans une configuration fixée de LeNet-5 adaptée, quelles performances sont rapportées sur AMHCD et comment les résultats varient-ils lorsque l'optimisation utilise Adam ou SGD avec momentum ?* Le protocole met en concurrence deux configurations complètes, car les taux d'apprentissage diffèrent également.

Trois dimensions de lecture sont retenues : la qualité globale de la classification, la répartition des erreurs entre classes et le comportement de l'apprentissage pendant les époques. La première dimension est quantifiée par des métriques sur le test ; les deux autres sont décrites grâce aux figures mises à disposition dans le rapport source.

E. Limites des comparaisons disponibles

L'étude ne contient ni mesure chronométrée des temps d'entraînement ni mesure de consommation mémoire. Elle ne permet donc pas de quantifier le coût opérationnel exact des optimiseurs. Les figures d'évolution sont disponibles pour Adam, tandis que la matrice de confusion conservée porte aussi sur Adam. Il serait incorrect d'interpréter ces seules visualisations comme des mesures comparatives par classe pour SGD.

IV. Données et méthodologie

A. Base AMHCD

La base *Amazigh Handwritten Character Database* (AMHCD) rassemble 25 740 images de caractères amazighs réparties en 33 catégories [3], [6]. Les échantillons du rapport étudié sont fournis sous forme d'images couleur, et les exemples d'une même classe illustrent déjà une variabilité de tracés, de contraste et de style d'écriture (Fig. 2).

Le partitionnement stratifié, réalisé au niveau des images, produit 18 018 exemples d'entraînement (70 %), 3 861 exemples de validation (15 %) et 3 861 exemples de test (15 %). La stratification vise à conserver la représentation des classes dans chaque sous-ensemble [7].

B. Prétraitement

Toutes les images sont converties en RGB, redimensionnées à 32×32 pixels puis normalisées, canal par canal, avec une moyenne et un écart-type de 0,5, ce qui place approximativement les valeurs transformées dans $[-1, 1]$ [8]. Pendant l'entraînement uniquement, une rotation aléatoire de $\pm 10^\circ$ augmente la diversité des orientations observées. Aucune augmentation stochastique n'est indiquée pour la validation ou le test.

C. Architecture convolutive

Le réseau suit l'enchaînement convolution–activation Tanh–pooling moyen propre à une variante classique de LeNet-5. La première convolution reçoit trois canaux au lieu d'un. La couche de sortie comporte 33 neurones, correspondant aux catégories de la base.

D. Configuration d'entraînement

Les convolutions successives comportent 6, 16 puis 120 filtres 5×5 ; deux opérations de pooling moyen 2×2 réduisent les dimensions spatiales. Après aplatissement, une couche dense à 84 unités précède la classification à 33 catégories. Le modèle comprend **63 961 paramètres entraînables**. Les sorties brutes (*logits*) sont exploitées par la perte d'entropie croisée, sans couche Softmax explicite pendant l'apprentissage.

Chaque variante est entraînée pendant 20 époques sur CPU, avec des lots de 64 images. Le meilleur état est sélectionné selon la perte de validation. Les graines pseudo-aléatoires de Python, NumPy et PyTorch sont initialisées à 42 afin de favoriser la reproductibilité [9]. L'implémentation est réalisée avec PyTorch [5].

Dimension	Choix du protocole	Conséquence pour l'analyse
Entrée	Images RGB normalisées, 32×32	Trois canaux traités directement par C1
Sortie	33 classes manuscrites	Classification multiclasse par entropie croisée
Comparaison	Adam contre SGD avec momentum	Comparaison de deux réglages d'optimisation complets
Interprétation	Mesures globales et visualisations	Analyse des erreurs sans nouvelle expérience

TABLE I
CADRE D'ANALYSE DE L'ÉTUDE, REFORMULÉ À PARTIR DU PROTOCOLE EXPÉRIMENTAL FOURNI.

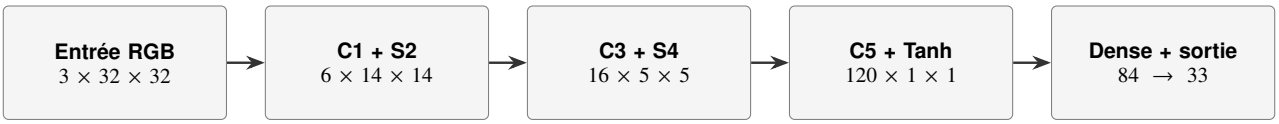


FIGURE 1. Structure du LeNet-5 adapté. Les convolutions sont suivies d'une activation Tanh ; les sous-échantillonnages utilisent AvgPool 2×2. Schéma reconstruit à partir de l'architecture décrite dans le rapport.

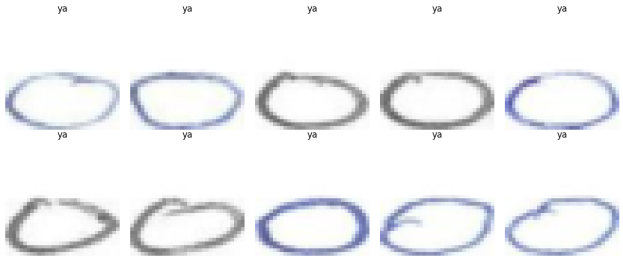


FIGURE 2. Exemples de variations manuscrites de la classe *ya* dans AMHCD, extraits des données figurant dans le rapport source.

E. Paramètres comparés

La comparaison oppose deux réglages d'optimisation : Adam utilise $\eta = 0{,}001$; SGD utilise $\eta = 0{,}01$ et un momentum de 0,9. La fonction objectif est identique dans les deux cas : l'entropie croisée multiclasse.

V. ANALYSE DÉTAILLÉE DE L'ARCHITECTURE

A. Calcul des dimensions spatiales

Pour une convolution sans remplissage, de noyau $K = 5$ et de pas unitaire, la taille spatiale est transformée selon $H_{\text{sortie}} = H_{\text{entree}} - K + 1$. La première opération transforme donc 32×32 en 28×28. Un pooling moyen 2×2 de pas 2 réduit ensuite la taille à 14×14.

Le même schéma produit successivement 10×10 après la deuxième convolution et 5×5 après le deuxième pooling. La convolution C5 applique enfin 120 filtres couvrant toute la zone 5×5, ce qui génère un tenseur $120 \times 1 \times 1$. Après aplatissement, la couche F6 contient 84 neurones, puis la couche de sortie produit 33 scores bruts.

B. Nombre de paramètres

Pour une convolution ordinaire avec biais, la quantité de coefficients entraînables est $(K^2 C_{\text{entree}} + 1)C_{\text{sortie}}$. Pour une couche dense, elle vaut $(N_{\text{entree}} + 1)N_{\text{sortie}}$. En appliquant ces relations aux tailles décrites dans le rapport, on retrouve le total annoncé de **63 961 paramètres**. Le tableau II expose ce calcul couche par couche. Les pools moyens et les fonctions Tanh ne possèdent aucun poids entraînable.

C. Rôle des non-linéarités

La transformation $\tanh(x)$ borne les activations entre -1 et 1. Les convolutions combinées à cette fonction créent des réponses non linéaires, tandis que le pooling moyen condense l'information locale. L'utilisation de Tanh et d'AvgPool correspond au protocole de l'étude et reste proche de l'esprit du réseau historique. Une comparaison avec ReLU ou MaxPool exigerait de nouveaux entraînements et ne peut être déduite des résultats actuels.

D. Fonction objectif et décision finale

Pour une image de classe y et les scores $z_1, \ldots, z_{33}$, la perte d'entropie croisée s'écrit :

$$\mathcal{L}(z, y) = -\log\left(\frac{\exp(z_y)}{\sum_{j=1}^{33} \exp(z_j)}\right). \tag{1}$$

La formulation logicielle `CrossEntropyLoss` combine implicitement le calcul du *log-softmax* et celui de la perte. Il n'est donc pas nécessaire d'ajouter une couche Softmax avant le calcul de l'erreur. L'étiquette prédite correspond à l'indice du score maximal.

E. Deux règles d'ajustement des poids

Adam adapte les pas de mise à jour à partir d'estimations des moments du gradient, alors que SGD avec momentum accumule une direction de déplacement. Dans l'étude, Adam utilise un taux d'apprentissage de 10^{-3} et SGD un taux de 10^{-2} avec momentum 0,9. Ces méthodes poursuivent la minimisation de la même perte, mais avec des trajectoires d'optimisation différentes. Les valeurs retenues sont celles documentées dans le rapport ; aucune recherche exhaustive d'hyperparamètres n'est décrite.

VI. PROTOCOLE D'ÉVALUATION

Chaque modèle est ajusté à partir du même découpage en entraînement, validation et test. À chaque époque, la perte et l'exactitude sont suivies sur la validation. L'évaluation finale

Bloc	Opération / noyau	Sortie (canaux, H, W)	Poids + biais
Entrée	RGB	(3, 32, 32)	—
C1	Conv 5×5, $3 \to 6$ + Tanh	(6, 28, 28)	456
S2	AvgPool 2×2	(6, 14, 14)	0
C3	Conv 5×5, $6 \to 16$ + Tanh	(16, 10, 10)	2 416
S4	AvgPool 2×2	(16, 5, 5)	0
C5	Conv 5×5, $16 \to 120$ + Tanh	(120, 1, 1)	48 120
F6	Dense $120 \to 84$ + Tanh	(84)	10 164
Sortie	Dense $84 \to 33$	(33)	2 805
Total			**63 961**

TABLE II
DIMENSIONS ET PARAMÈTRES DE LENET-5 ADAPTÉ. LES NOMBRES DE PARAMÈTRES SONT CALCULÉS À PARTIR DES COUCHES INDIQUÉES DANS LE RAPPORT (ILS NE CORRESPONDENT PAS À UN NOUVEL ENTRAÎNEMENT).

est réalisée une fois le meilleur modèle choisi, sur les 3 861 images de test qui n'ont pas servi à la sélection.

Pour une classe k, la précision P_k, le rappel R_k et le score $F_{1,k}$ sont obtenus à partir des vrais positifs, faux positifs et faux négatifs. Les métriques macro sont des moyennes non pondérées sur les 33 catégories :

$$P_{\text{macro}} = \frac{1}{33}\sum_{k=1}^{33} P_k, \quad R_{\text{macro}} = \frac{1}{33}\sum_{k=1}^{33} R_k. \tag{2}$$

$$F_{1,\text{macro}} = \frac{1}{33}\sum_{k=1}^{33} \frac{2P_kR_k}{P_k + R_k}. \tag{3}$$

L'exactitude (*accuracy*) mesure la fraction totale d'images correctement classées. Une matrice de confusion ainsi que des exemples de prédictions et d'erreurs complètent ces mesures.

A. Séparation des usages de données

Les images d'entraînement servent à ajuster les poids du CNN ; la validation sert à choisir, pour chaque optimiseur, l'état présentant la perte la plus faible. Le test est réservé à l'estimation finale des performances. En particulier, le score de test ne doit pas être utilisé pour ajuster les paramètres du modèle. Cette séparation limite le risque de surestimation lié à la sélection sur les exemples évalués.

Le rapport indique une stratification par classe, mais ne documente pas de division par identité de rédacteur. Il est donc impossible d'affirmer, à partir des informations disponibles, que les personnes ayant écrit les images du test sont toutes absentes de l'entraînement. Cette nuance est importante pour l'interprétation d'une généralisation à de nouveaux scripteurs.

B. Macro-moyenne et exactitude

L'exactitude favorise implicitement les catégories les plus représentées lorsqu'un jeu est déséquilibré. Les moyennes macro attribuent au contraire le même poids numérique à chaque classe. Elles sont donc complémentaires. En outre, le F1 macro présenté au Tableau III est calculé en moyennant les $F_{1,k}$ des classes : il ne faut pas le confondre avec la moyenne harmonique de P_{macro} et de R_{macro}.

VII. RÉSULTATS EXPÉRIMENTAUX

A. Comparaison globale des optimiseurs

Les deux configurations dépassent 97 % d'exactitude. SGD avec momentum obtient 97,95 % contre 97,38 % pour Adam, soit un écart absolu de **0,57 point de pourcentage**. L'ordre reste identique pour les métriques macro (Tableau III). Ces valeurs décrivent la comparaison dans les conditions expérimentales rapportées et ne constituent pas, à elles seules, un test de significativité statistique.

Les courbes disponibles concernent Adam (Fig. 3). Elles montrent une baisse régulière de la perte et une augmentation de l'exactitude d'entraînement et de validation durant les 20 époques. Les écarts visuels entre entraînement et validation demeurent modérés. Le rapport ne fournit pas de courbes SGD comparables ; aucune conclusion sur sa vitesse de convergence ne peut donc être quantifiée ici.

B. Prédictions qualitatives

La galerie de prédictions (Fig. 4) illustre des caractères correctement reconnus malgré des morphologies et des colorations variées. Cette sélection visuelle atteste que certaines variations intra-classe sont prises en compte, mais elle ne remplace pas les métriques agrégées obtenues sur la totalité du test.

La matrice de confusion de la configuration Adam est analysée à la page suivante. Elle permet de replacer ces observations qualitatives dans l'ensemble des 33 classes et d'identifier les erreurs hors diagonale.

VIII. LECTURE CRITIQUE DES RÉSULTATS GLOBAUX

A. Écart absolu et ampleur pratique

La comparaison du Tableau III indique un gain d'exactitude de 97,95 – 97,38 = **0,57** point de pourcentage pour SGD avec momentum. Les scores de précision, rappel et F1 macro sont

Optimiseur	Exactitude	Précision macro	Rappel macro	F1 macro
Adam	97,38 %	0,9743	0,9738	0,9739
SGD + momentum	**97,95 %**	**0,9798**	**0,9795**	**0,9795**

TABLE III
PERFORMANCES RAPPORTÉES SUR LE JEU DE TEST AMHCD (3 861 IMAGES). TOUTES LES VALEURS PROVIENNENT DU RAPPORT FOURNI.

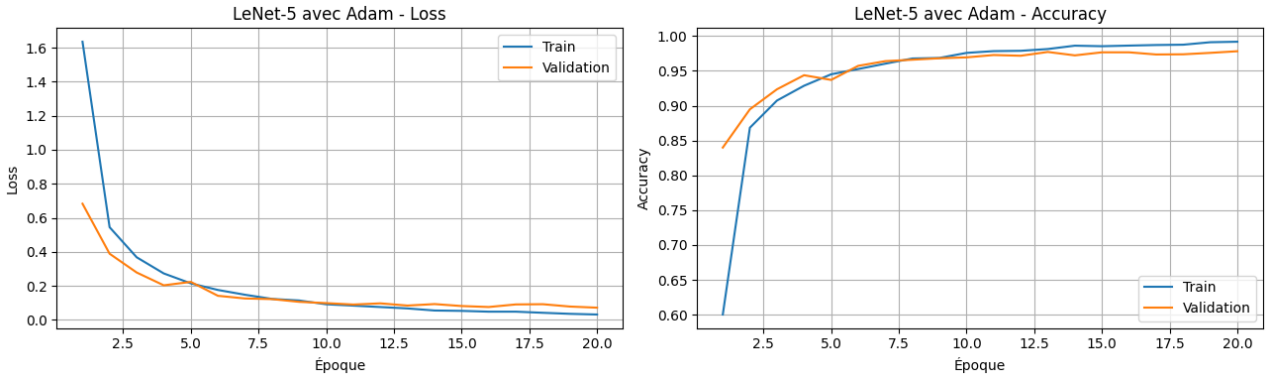


FIGURE 3. Évolution de la perte et de l'exactitude avec Adam sur 20 époques : entraînement (bleu) et validation (orange). Figure reprise du rapport source.

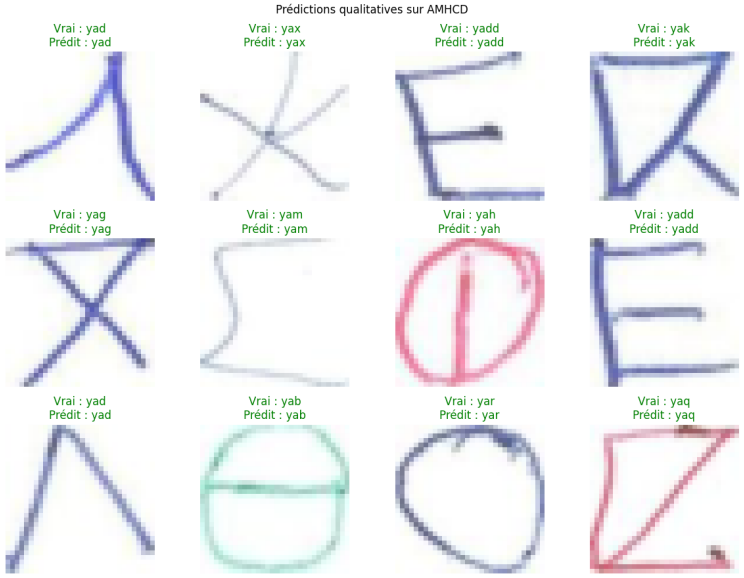


FIGURE 4. Exemples de prédictions qualitatives du modèle sur AMHCD (vrai label et label prédit), d'après le rapport initial.

également légèrement plus élevés pour ce réglage. Le même sens de variation pour plusieurs indicateurs réduit le risque que le classement provienne d'une seule métrique atypique, sans constituer une preuve statistique.

Sur les 3 861 images du test, une exactitude arrondie à 97,95 % correspond *approximativement* à 79 erreurs, contre environ 101 à 97,38 %. Ces ordres de grandeur sont calculés à partir de pourcentages publiés à deux décimales et ne remplacent pas un comptage direct des prédictions individuelles. Ils aident simplement à interpréter la taille de l'écart sur ce test précis.

B. Ce qu'indiquent les scores macro

Pour Adam, le F1 macro est de 0,9739 ; pour SGD, il est de 0,9795. Ces valeurs indiquent que les performances moyennes calculées sur les classes sont élevées. Elles ne permettent toutefois pas de savoir si un petit nombre de caractères difficiles concentre l'essentiel des échecs. Pour cette question, une inspection de la matrice de confusion et des scores par classe serait nécessaire.

C. Pourquoi la convergence ne suffit pas

Les courbes de la Fig. 3 suggèrent une bonne progression pendant les 20 époques pour Adam : la perte diminue et l'exactitude augmente. Cela documente le comportement d'une configuration, mais ne justifie pas une affirmation sur la vitesse de SGD, pour lequel les courbes ne sont pas montrées dans le document d'origine. Un indicateur de convergence devrait être comparé à budget d'époques, nombre d'itérations et critères d'arrêt identiques.

D. Comparabilité des optimiseurs

Deux facteurs changent entre les entraînements : l'algorithme d'optimisation et le taux d'apprentissage. La supériorité ponctuelle de SGD observée ici pourrait être sensible à ces réglages, à la graine et à la composition du test. Pour isoler l'effet du seul optimiseur, un protocole complémentaire devrait étudier plusieurs taux d'apprentissage par méthode et répéter l'entraînement sur différentes graines.

E. Interprétation avec prudence

Dans ce protocole, le résultat le plus élevé n'est pas synonyme de supériorité universelle de SGD. Une seule expérience par configuration ne documente ni la variance entre répétitions ni un intervalle de confiance. La comparaison garde néanmoins une valeur expérimentale : elle montre qu'une variante compacte de LeNet-5, avec des réglages simples, peut atteindre une performance élevée sur le découpage AMHCD considéré.

F. Informations utiles à une reproduction

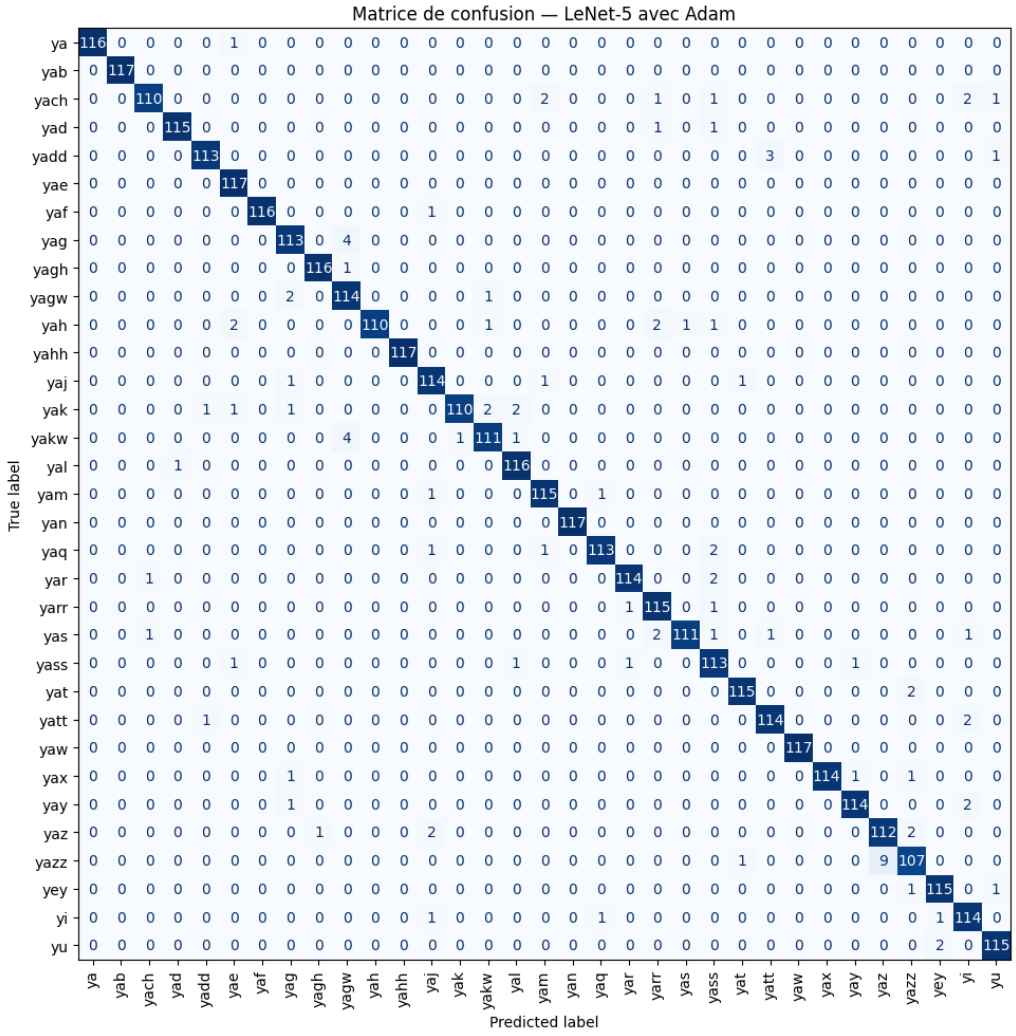
Le rapport précise la graine globale, les tailles de lots, le nombre d'époques, les transformations, le partitionnement, la fonction de perte et les taux d'apprentissage. La version exacte de Python et celle de PyTorch restent cependant annoncées comme à vérifier dans le document source. Une reproduction complète gagnerait à consigner les versions installées, les fichiers de partition et les états de modèle retenus.

IX. Analyse des confusions

A. Matrice de confusion

La matrice de confusion disponible pour Adam présente une diagonale dominante, cohérente avec l'exactitude de 97,38 %. Les cellules hors diagonale regroupent un nombre plus limité de confusions. Certaines erreurs concernent des étiquettes de caractères morphologiquement proches, telles que *ya*/*yab* ou *yat*/*yatt* d'après les commentaires du rapport original.

La figure 5 représente les résultats par classe pour Adam uniquement : une matrice correspondante pour SGD n'était pas incluse dans le document source. Cette restriction empêche de comparer directement, caractère par caractère, les profils d'erreur des deux optimiseurs.



Indicateur	Adam	SGD	Interprétation
Exactitude	97,38 %	97,95 %	Écart observé : +0,57 point pour SGD
F1 macro	0,9739	0,9795	Écart observé : +0,0056
Erreurs estimées*	~ 101	~ 79	Calcul approximatif sur 3 861 tests

TABLE IV
LECTURE SYNTHÉTIQUE DES RÉSULTATS DE TEST. *ESTIMATION CALCULÉE À PARTIR DES EXACTITUDES ARRONDIES ET NON COMPTAGE ORIGINAL DES ERREURS.

FIGURE 5. Matrice de confusion du modèle LeNet-5 avec Adam sur l'ensemble de test. Les valeurs diagonales correspondent aux prédictions correctes. Figure issue du rapport source.

X. INTERPRÉTATION ET DISCUSSION

A. Erreurs de classification

Les erreurs illustrées dans la Fig. 6 concernent notamment *yaz*/*yazz*, *yadd*/*yatt*, *yad*/*yass* et *yat*/*yi*. Ces exemples montrent que de petites différences de structure, de fermeture ou de disposition des traits peuvent modifier l'étiquette prédite.

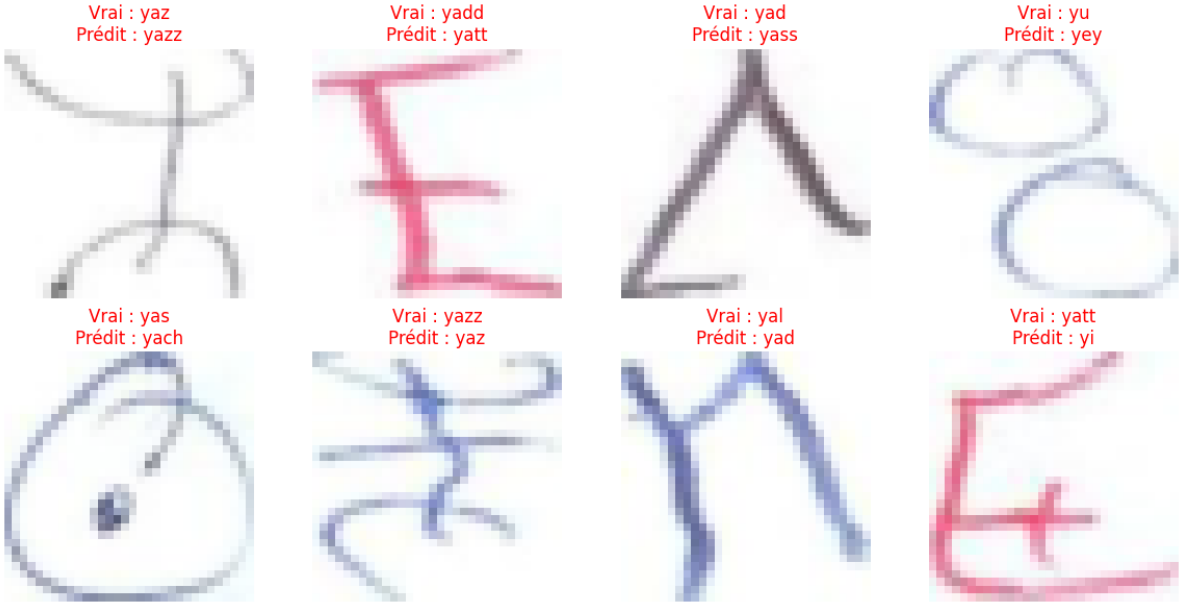


FIGURE 6. Exemples de mauvaises classifications : étiquette réelle et prédiction, repris du rapport source.

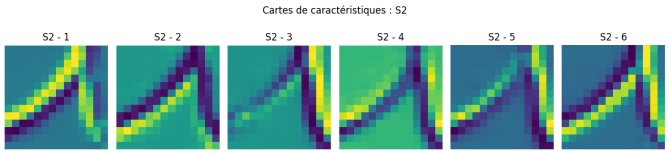


FIGURE 7. Cartes d'activation de la couche S2 de LeNet-5 sur un caractère test, d'après le rapport source.

B. Cartes de caractéristiques

Les six cartes de la couche S2 (Fig. 7) révèlent des réponses différentes à la forme d'entrée. Elles illustrent l'extraction de motifs locaux par les filtres, sans permettre à elles seules d'attribuer une fonction sémantique certaine à chaque canal.

C. Limites et pistes de recherche

La comparaison porte sur un découpage de données et une graine (42), sans répétitions ni intervalles de confiance. De plus, le taux d'apprentissage varie en même temps que l'optimiseur : le gain de 0,57 point observé pour SGD doit donc être interprété comme un résultat propre à cette configuration, non comme une preuve de supériorité systématique. Le document source ne précise pas non plus si les images des mêmes scripteurs sont isolées entre entraînement et test.

La conservation du format RGB est une adaptation technique, mais le protocole ne comprend aucune comparaison contrôlée avec des images en niveaux de gris. Il ne permet donc pas de démontrer un apport spécifique de la couleur. Enfin, la généralisation à d'autres collections ou dispositifs d'acquisition reste à établir.

Les prolongements prioritaires sont : répéter les entraîne-

ments avec plusieurs graines, optimiser équitablement le taux d'apprentissage pour chaque méthode, présenter les confusions par classe pour les deux modèles et étudier un découpage indépendant des scripteurs lorsque les identifiants existent. Des comparaisons avec une version monochrome, des architectures modernes ou des augmentations mieux ciblées apporteraient ensuite des éléments supplémentaires.

D. Évaluation de la robustesse à l'acquisition

Les images AMHCD sont redimensionnées à une résolution relativement faible. Dans ces conditions, un caractère ne contient parfois que quelques pixels utiles pour distinguer deux graphèmes. La rotation utilisée pendant l'entraînement contribue à familiariser le modèle avec les variations d'orientation, mais les résultats rapportés ne renseignent pas sur son comportement lorsque le contraste change fortement, lorsqu'une partie du trait est masquée ou lorsque le fond présente du bruit. Un prolongement pertinent consisterait à construire des jeux de test perturbés de manière contrôlée, en conservant les mêmes étiquettes et en séparant strictement ces transformations de la phase d'entraînement. Les performances pourraient ensuite être rapportées en fonction de l'amplitude de chaque perturbation, et non réduites à une seule exactitude globale. Il faudrait néanmoins veiller à ne pas transformer la forme d'un caractère au point de changer sa véritable catégorie.

E. Incertitude sur la comparaison Adam–SGD

L'écart de 0,57 point de pourcentage en faveur de SGD est un résultat descriptif sur un découpage donné. Pour en évaluer la stabilité, il conviendrait de répéter les entraînements avec plusieurs initialisations, puis de rapporter la moyenne et la dispersion des scores de test. Si les prédictions individuelles des deux modèles sur les mêmes images étaient conservées, une comparaison appariée, par exemple un test de McNemar ou un rééchantillonnage des différences, permettrait de mieux caractériser l'incertitude sur l'écart observé. De telles analyses nécessitent les sorties détaillées des modèles ; elles ne peuvent pas être reconstituées fidèlement à partir des seules exactitudes agrégées du rapport. La conclusion présente donc SGD comme le meilleur réglage *dans l'expérience documentée*, sans généralisation statistique non justifiée.

F. Conditions de reproduction et traçabilité

Le maintien d'une graine pseudo-aléatoire identique facilite la reprise de l'expérience, mais ne suffit pas à assurer une répétition bit à bit sur tous les systèmes. Les versions des bibliothèques, les routines de prétraitement, l'ordre des exemples et la configuration matérielle peuvent affecter l'exécution. Pour une étude ultérieure, les éléments à conserver seraient notamment les identifiants des images présentes dans chaque partition, le script d'entraînement, les hyperparamètres complets, la courbe de chaque optimiseur et le fichier de poids sélectionné par la validation. Les versions exactes de Python et de PyTorch sont indiquées comme restant à vérifier dans le document de départ : elles ne sont donc pas spécifiées ici comme des faits établis. Une telle documentation faciliterait l'audit des résultats et la comparaison de nouvelles variantes sur les mêmes exemples.

G. Intérêt et limites d'une architecture compacte

Avec 63 961 paramètres entraînables, la variante de LeNet-5 étudiée reste modeste par rapport à de nombreuses architectures contemporaines. Cette taille rend plausible son usage dans un environnement où le calcul et la mémoire sont limités, comme l'ordinateur CPU employé pour l'étude. Elle ne permet toutefois pas de déduire directement le temps d'inférence, l'utilisation effective de mémoire ou les besoins énergétiques : aucun benchmark matériel détaillé n'est fourni. Une comparaison de déploiement nécessiterait des mesures chronométrées sur le même équipement, à plusieurs tailles de lots et en précisant le coût du prétraitement. Dans le cadre présent, l'apport démontré concerne principalement la qualité de classification du protocole, et non une supériorité vérifiée en vitesse ou en consommation.

XI. Conclusion et perspectives

Une variante RGB de LeNet-5 à 63 961 paramètres a été évaluée sur 33 classes de caractères manuscrits amazighs. Les résultats rapportés atteignent 97,95 % d'exactitude avec SGD et 97,38 % avec Adam sur le jeu de test. L'architecture compacte s'avère ainsi efficace dans le protocole considéré, malgré des confusions persistantes entre caractères proches.

La poursuite de l'étude devrait examiner plusieurs graines, la robustesse par auteur lorsque les métadonnées le permettent, l'influence des augmentations d'images et des architectures plus profondes (ResNet, VGG), ainsi que l'évaluation des confusions pour chaque optimiseur.

Références

[1] Y. LeCun, L. Bottou, Y. Bengio et P. Haffner, « Gradient-Based Learning Applied to Document Recognition », *Proceedings of the IEEE*, vol. 86, n° 11, p. 2278–2324, 1998. DOI : 10.1109/5.726791.

[2] I. Goodfellow, Y. Bengio et A. Courville, *Deep Learning*, MIT Press, 2016.

[3] Y. Es Saady, A. Rachidi, M. El Yassa et D. Mammass, « AMHCD : A Database for Amazigh Handwritten Character Recognition Research », *International Journal of Computer Applications*, vol. 27, n° 4, p. 44–48, 2011.

[4] M. Benaddy, O. El Meslouhi, Y. Es-saady et M. Kardouchi, « Handwritten Tifinagh Characters Recognition Using Deep Convolutional Neural Networks », *Sensing and Imaging*, vol. 20, art. 9, 2019.

[5] A. Paszke et al., « PyTorch : An Imperative Style, High-Performance Deep Learning Library », *NeurIPS*, 2019.

[6] M. Benaddy, « Amazigh Handwritten Character Database (AMHCD) », Kaggle, Kaggle : AMHCD, consulté le 13 oct. 2026.

[7] Scikit-learn, « train_test_split », documentation officielle, Documentation scikit-learn.

[8] PyTorch, « torchvision.transforms.Normalize », documentation officielle, Documentation PyTorch.

[9] PyTorch, « Reproducibility », documentation officielle, Documentation PyTorch.